

VibeThinker-3B-Agent: Grafting Tool Calling and Agentic Reasoning onto a Frontier Small Reasoning Model via Composed Reinforcement Learning

[Abu Ahmad-2121725042] . [Farshid Abrar Labib-2211809042]

CSE465 — Course Project Report

Department of Electrical and Computer Engineering, North South University · Summer 2026

Abstract

Small language models have recently crossed a capability threshold that was long assumed to require hundreds of billions of parameters. VibeThinker-3B, a 3B-parameter dense model post-trained under the Spectrum-to-Signal paradigm, reaches 94.3 on AIME26 and 80.2 Pass@1 on LiveCodeBench v6 — inside the performance band of frontier reasoning systems that are two orders of magnitude larger. The model nevertheless ships with an explicit limitation: it was never trained on tool-calling or agentic data, and its authors advise against using it for function calling or autonomous workflows. This report designs an upgrade path, VibeThinker-3B-Agent, that adds precisely these two capabilities — (i) structured tool calling and (ii) a multi-turn agentic reasoning loop — while protecting the compact reasoning core that makes the base model valuable. Rather than inventing new machinery, we compose five published, open-source reinforcement learning mechanisms, each of which resolves one specific obstacle we identify: Tool-N1’s pure-RL binary-reward recipe teaches tool-call syntax and selection without a weight-destructive supervised stage; Search-R1’s observation-token masking, GiGPO’s critic-free step-level credit assignment, and ARPO’s entropy-guided rollout branching make the multi-turn loop trainable and affordable at 3B scale; and ReTool’s code-interpreter setting keeps every reward verifiable. Because these mechanisms act at five orthogonal levels of the policy-gradient update — rollout, prompt, advantage, loss, and reward — they collapse into a single objective that retains VibeThinker’s own MGPO algorithm unchanged. A regression guard re-evaluates the mathematics, coding, and instruction-following suites after every training phase, making reasoning retention a first-class experimental result rather than an afterthought. The full plan runs on the open verl stack and is scoped to a two-member course project.

1 Introduction

Reinforcement learning with verifiable rewards has reshaped what small language models can do. The clearest recent evidence is VibeThinker-3B [1], a 3-billion-parameter dense model built on the Qwen2.5-Coder-3B base [13] and post-trained through curriculum SFT, multi-domain reinforcement learning with MaxEnt-Guided Policy Optimization (MGPO), offline self-distillation, and instruction RL. On competition mathematics and executable coding — tasks whose answers can be checked mechanically — it matches or exceeds flagship systems such as DeepSeek V3.2 (671B) and Kimi K2.5 (1T). Its authors frame this with a Parametric Compression-Coverage Hypothesis: verifiable reasoning compresses into a compact core, while broad world knowledge does not.

The same model card, however, draws a hard boundary. VibeThinker-3B was trained on no tool-calling or agent data whatsoever, and its authors explicitly recommend against using it for function calling, API orchestration, or autonomous coding agents. In 2026 this is a serious practical gap: modern reasoning systems are increasingly judged not only by what they can derive in a single forward pass, but by how well they operate a loop — calling a search engine, executing code, reading the result, and revising their plan. A model that reasons at frontier level but cannot act is an engine without a drivetrain.

Closing this gap at 3B scale is not a matter of prompting. Because VibeThinker descends from a base (not instruct) checkpoint, it has no trained tool-call surface to prompt against; because its headline value is an exceptionally optimized reasoning core, every additional training stage risks eroding exactly the numbers that justify the project; and because multi-turn agentic RL inserts externally generated observation tokens into trajectories, a naive application of the model’s own MGPO objective would corrupt the policy gradient. We identified eight such constraints spanning format grounding, catastrophic forgetting, context budget, credit assignment, rollout cost, reward hacking, and reward verifiability.

Our central design decision is composition over invention. For each constraint there now exists a published, open-source mechanism that resolves it, and — crucially — these mechanisms act at different levels of the RL update, so they can be stacked without interference. This report contributes:

- **A constraint analysis** that maps every obstacle to grafting tool use onto a small reasoning model to a specific, citable solution, grounded in a close reading of VibeThinker-3B and five agentic-RL papers.
- **A composed training objective** that unifies Tool-N1’s binary reward, Search-R1’s observation masking, GiGPO’s step-level advantages, and ARPO’s entropy-based branching inside VibeThinker’s own MGPO framework — a combination none of the source papers explores.
- **A two-phase, regression-guarded pipeline** (single-step tool calling first, multi-turn agentic RL second) in which reasoning retention is measured after every phase and any excessive drop rolls the phase back.
- **A falsifiable scaling hypothesis:** prior work reports that tool-calling RL yields limited gains below 7B, but its 3B evidence comes from a generic instruct model. We conjecture the true predictor is the backbone’s reasoning quality, not its parameter count — and VibeThinker-3B is the cleanest possible instrument to test this.

The remainder of this report reviews the literature we build on (Section 2), presents the methodology with its two visual summaries (Section 3), lays out the experimental design (Section 4), and concludes (Section 5).

2 Related Works

2.1 Small Reasoning Models

DeepSeek-R1 [8] established that rule-based RL over verifiable rewards — built on the group-relative policy optimization (GRPO) objective introduced in DeepSeekMath [9] — can elicit long-horizon reasoning without step-level supervision. The VibeThinker series pushes this recipe into the

strictly-small regime. The 1.5B model demonstrated feasibility; the 3B successor [1] adds curriculum two-stage SFT, a single 64K-context RL stage, a Long2Short efficiency stage, offline self-distillation, and instruct RL, and lands inside the frontier band on AIME25/26, HMMT25, and LiveCodeBench v6. Two of its design choices matter directly for us: MGPO weights training prompts by their proximity to the model’s capability boundary (empirical accuracy near 0.5), and all RL is run strictly on-policy for stability. Notably, the model’s documented weaknesses — knowledge-heavy tasks (GPQA-Diamond 70.2) and the complete absence of tool training — are precisely the two things an agentic upgrade addresses: external tools are, in effect, rentable coverage.

2.2 Reinforcement Learning for Single-Step Tool Calling

Nemotron-Research-Tool-N1 [2] is the anchor for our first feature. It trains Qwen2.5-Instruct backbones to emit <think> reasoning followed by a <tool_call> JSON block, using GRPO with a strictly binary reward: 1 if the output format is correct and the predicted call exactly matches the ground-truth name and arguments (as parsed dictionaries, so argument order is free), 0 otherwise. Three of its findings shape our design. First, the binary reward beats fine-grained partial credit (84.82 vs. 82.58 BFCL average), because partial credit invites reward hacking. Second, pure RL matches or beats the SFT-then-RL pipeline under equal data budgets (83.24 vs. 83.17), which lets us delete the riskiest stage — supervised fine-tuning — from our plan entirely. Third, no tool is ever executed during training: the reward is a dictionary comparison against labeled calls from xLAM [10] and ToolACE [11], which makes training cheap. Tool-N1’s stated limitation is scope: it trains and evaluates single-step prediction only, and its scaling study attributes weak sub-7B results to backbone quality — an attribution our project directly probes.

2.3 Multi-Turn Agentic Reinforcement Learning

Four lines of work supply the machinery Tool-N1 lacks. Search-R1 [3] trains an LLM to interleave reasoning with live search queries and introduces the practice we consider non-negotiable: masking retrieved observation tokens out of the policy-gradient loss, without which multi-turn RL destabilizes. It is validated on Qwen2.5-3B — the existence proof that multi-turn tool RL works at our scale. GiGPO [4] addresses credit assignment: trajectory-level rewards give every turn the same advantage, so it extends group-relative comparison to the step level via anchor-state grouping, critic-free and memory-neutral — properties that matter on course-project compute. ARPO [5] observes that token entropy spikes immediately after tool feedback and branches rollouts adaptively at those points, matching trajectory-level baselines with roughly half the tool-call budget. ReTool [6] trains code-interpreter use on competition mathematics specifically, showing that tool-integrated reasoning can be learned with purely outcome-based rewards in a domain where every answer is checkable. Finally, Context-Folding [7] teaches agents to collapse completed sub-trajectories into summaries, cutting active context roughly tenfold on long-horizon tasks; we cite it as the designated solution to context growth but scope it out of the build (Section 3.3). No prior work composes these mechanisms, and none applies any of them to a backbone whose per-parameter reasoning density approaches VibeThinker’s.

3 Methodology

3.1 Problem Formulation

Let $\pi\theta$ denote the VibeThinker-3B policy and $Z = \{z_i\}$ a registry of tools, each a triple of name, description, and parameter schema. At turn t the policy conditions on the context c_t (the query, its own prior reasoning and calls, and all tool observations $o_1..o_{t-1}$) and emits either a tool call $a_t \subseteq Z$ or a final answer. Feature 2 (tool calling) is the single-step special case: given (query, Z), predict one correct a_1 . Feature 1 (agentic) is the general multi-turn case, where observations are produced by a real executor (a sandboxed Python interpreter) and the episode ends with a verifiable answer. The training problem is to maximize expected verifiable reward over this process without degrading $\pi\theta$'s existing single-pass reasoning distribution — the constraint that distinguishes this project from training a tool model from scratch.

3.2 The Two-Phase, Regression-Guarded Pipeline

Figure 1 shows the overall design. The pipeline deliberately mirrors the staged philosophy of VibeThinker's own post-training — acquire a capability, then consolidate it — but replaces every supervised stage with on-policy RL, following Tool-N1's Finding 2. A regression guard (AIME25/26, HMMT25, LiveCodeBench v6, GPQA-Diamond, IFEval) runs after every phase; a drop beyond a pre-registered tolerance rolls the phase back to its last acceptable checkpoint.

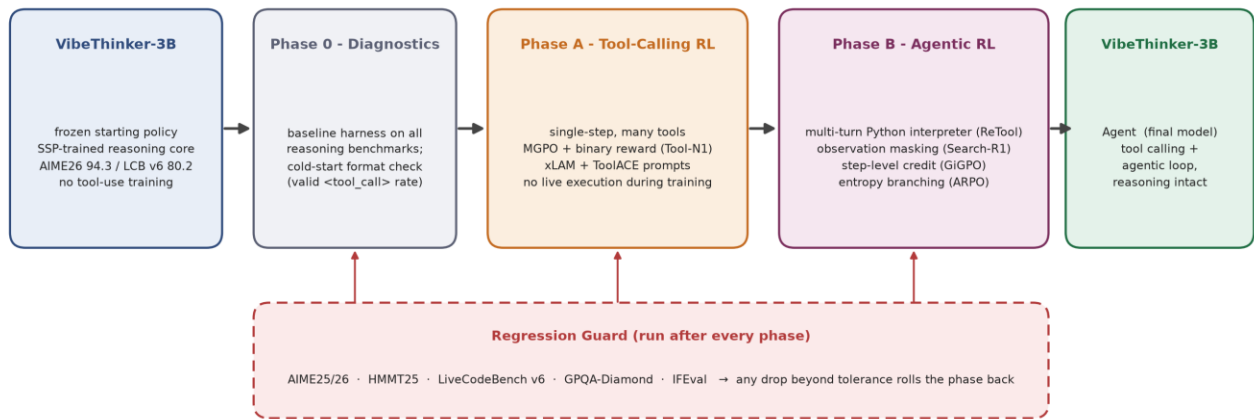


Figure 1: Overview of the proposed pipeline. VibeThinker-3B passes through diagnostics, single-step tool-calling RL (Phase A), and multi-turn agentic RL (Phase B); a regression guard re-evaluates the reasoning suites after every phase.

Phase 0 — Diagnostics. We first reproduce the base model's published scores to establish the regression baseline, then run a cold-start probe: 100 xLAM prompts through Tool-N1's thinking template, measuring the rate of syntactically valid `<tool_call>` outputs at step zero. This single number decides Phase A's first knob. Tool-N1 skipped SFT on backbones that already possessed a trained function-calling surface; VibeThinker descends from a base checkpoint and may not. If the valid-format rate is near zero, every rollout in a GRPO group scores 0, the group advantage is undefined, and no gradient flows.

Phase A — Single-Step Tool-Calling RL. We adopt Tool-N1's recipe wholesale — lightweight prompt template, binary format-plus-exact-match reward, xLAM and ToolACE data, no tool execution during training — with one substitution: the underlying GRPO objective is replaced by VibeThinker's MGPO, whose maximum-entropy prompt weighting $w(q)$ is a strict superset of GRPO and requires

no code beyond what the base model’s pipeline already implements. If the cold-start probe fails, we prepend the minimal viable intervention (a format-only reward for the first ~ 50 steps, escalating to a few hundred format-warmup examples only if necessary), then proceed with pure RL. The KL anchor ($\beta = 10^{-3}$, following Tool-N1) keeps the policy close to the frozen reference — our principal defense against forgetting.

Phase B — Multi-Turn Agentic RL. Starting from the Phase A checkpoint (format competence transfers), the model now reasons, emits Python inside `<tool_call>` tags, receives sandboxed execution output as an observation, and iterates until it commits a final boxed answer, which is checked by the same verifiers VibeThinker’s math RL already uses. We choose ReTool’s domain — tool-integrated competition mathematics — deliberately: it keeps rewards binary and verifiable, reuses the base model’s existing sandbox and verifier infrastructure, and aims the new capability directly at the model’s headline benchmark. Three mechanisms make this phase trainable at 3B on modest compute, and Figure 2 shows how they fit together.

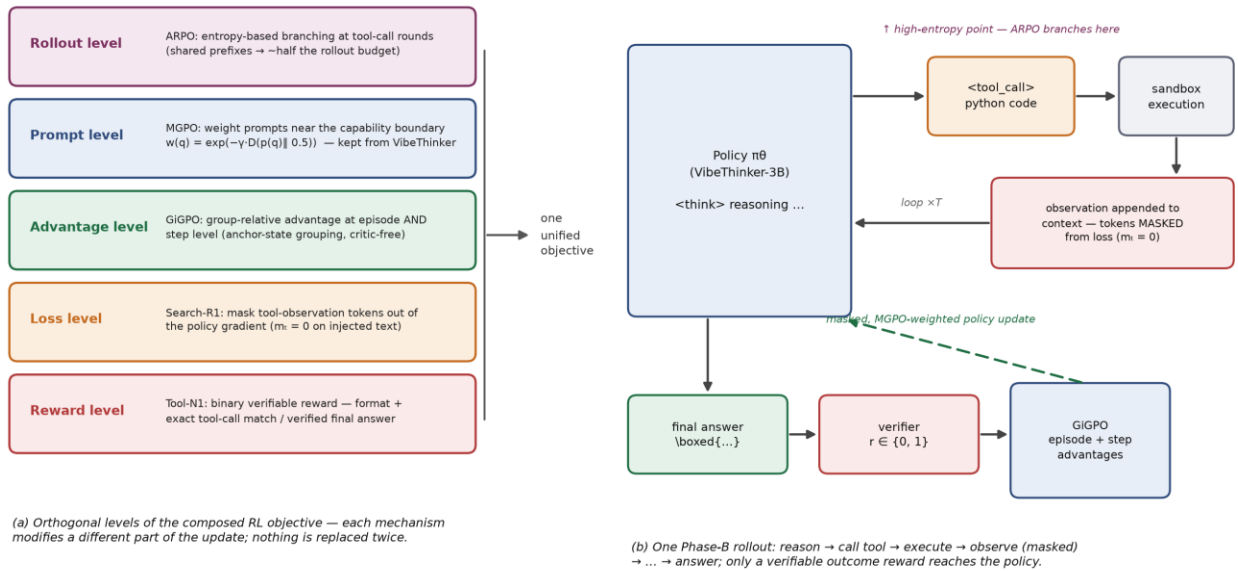


Figure 2: (a) The five orthogonal levels of the composed RL objective and the paper each level is borrowed from. (b) A single Phase-B rollout: observation tokens enter the context but are masked from the loss; only a verifiable outcome reward, shaped into episode- and step-level advantages, updates the policy.

The composed objective. Formally, the update is MGPO’s clipped surrogate with two modifications: a per-token mask m_t that zeroes the gradient on every observation token (Search-R1), and the group-relative advantage replaced by GiGPO’s blend of episode-level and anchor-state step-level advantages. Rollouts are generated with ARPO’s entropy-adaptive branching — sampling forks at high-entropy tool-call rounds and shares prefixes elsewhere — and rewards remain strictly binary throughout. Each borrowed mechanism occupies its own level of the update (Figure 2a): branching shapes how trajectories are sampled, $w(q)$ decides which prompts get gradient, GiGPO decides which turns get credit, the mask decides which tokens get gradient, and the reward decides what counts as success. Because the levels are orthogonal, nothing is replaced twice and no source paper’s guarantees are violated. This composition — and its execution on a frontier-grade small reasoning core — is the project’s methodological novelty.

3.3 Implementation Considerations

Forgetting. Beyond the KL anchor, we hold VibeThinker’s offline self-distillation stage in reserve: if Phase B measurably degrades Phase A’s tool-calling scores (or vice versa), verified trajectories from both specialist checkpoints are distilled into one student, reusing a mechanism the base model’s pipeline already contains rather than inventing a new consolidation step.

Context budget. VibeThinker generates up to $\sim 40\text{K}$ tokens and was RL-trained at a 64K window; tool-integrated mathematics fits comfortably, particularly since ReTool reports that interpreter access shortens reasoning traces. Long-horizon domains that would not fit (deep research, software engineering) are explicitly out of scope, with Context-Folding [7] cited as the designated extension.

Compute. Phase A trains without executing a single tool; Phase B’s dominant cost — on-policy rollouts with live execution — is roughly halved by ARPO’s branching and bounded further by response caching and small group sizes. All five source methods have open implementations on the verl RL stack [12], giving the project a single training framework end to end. Parameter-efficient (LoRA) variants of both phases serve as the fallback if full-parameter training exceeds our allocation.

Reward integrity. Phase A’s reward is a dictionary comparison and cannot be gamed by execution tricks. Phase B inherits the stricter surface of agentic settings; we sandbox execution without network access, keep held-out test answers outside the environment, and re-verify a sample of high-reward trajectories manually each run.

4 Experimental Design

The evaluation is organized around three questions: (Q1) does the upgraded model gain the two capabilities; (Q2) does it retain its reasoning; (Q3) does backbone reasoning quality — not parameter count — predict tool-RL gains at 3B. Table 1 maps each question to its benchmarks.

Question	Benchmarks	Compared against
Q1a · Tool calling	BFCL, API-Bank, ACEBench (single-turn)	Tool-N1-7B/14B; base model
Q1b · Agentic loop	AIME25/26, HMMT25 with interpreter access	Same model, pure CoT (no tools)
Q2 · Retention	AIME25/26, HMMT25, LiveCodeBench v6, GPQA-D, IFEval — after every phase	Published base-model scores
Q3 · Scaling claim	BFCL delta from Phase A	Tool-N1’s Qwen2.5-3B-Instruct point

Table 1: Evaluation matrix. Every training phase ends with the full Q2 suite; results are reported as deltas against the frozen baseline.

Planned ablations follow the composition levels of Figure 2a, removing one mechanism at a time from Phase B: (i) no observation masking, (ii) episode-only advantages instead of GiGPO, (iii) independent rollouts instead of ARPO branching, and (iv) uniform prompt weights instead of MGPO’s $w(q)$. Each ablation isolates the contribution of exactly one borrowed mechanism, which doubles as an empirical check that the levels really are orthogonal. We report tool-call budget alongside accuracy for (iii), since ARPO’s claim is efficiency at parity.

We deliberately report no projected numbers. The two pre-registered success criteria are: Phase A lifts BFCL by a clearly larger margin than Tool-N1 observed on generic Qwen2.5-3B-Instruct while

the Q2 suite stays within tolerance (≤ 2 points on every benchmark); and Phase B’s interpreter-augmented mathematics beats the same checkpoint’s pure-CoT scores. Either outcome, positive or negative, is informative for Q3 — a null result would indicate that reasoning density alone does not rescue small-model tool RL, which contradicts the backbone-quality attribution in the source paper.

5 Conclusion

This report set out to answer a concrete question: can the two capabilities VibeThinker-3B explicitly lacks — tool calling and agentic operation — be added without spending the reasoning capital that makes the model exceptional? Our answer is a design rather than a wish: every obstacle we identified maps to a published, open-source mechanism, the mechanisms occupy orthogonal levels of the policy-gradient update, and they therefore compose into a single MGPO-compatible objective executed in two regression-guarded phases. The scientific payoff extends beyond the artifact. If a 3B model whose only distinction is reasoning density gains disproportionately from tool RL, the field’s current scaling attribution for tool use needs revision; if tools meaningfully lift its knowledge-bound scores, the Compression-Coverage Hypothesis acquires its natural corollary — that coverage can be rented at inference time rather than stored in parameters. Both claims are falsifiable with course-project compute, which is precisely what makes this a viable CSE465 project rather than an aspiration.

References

- [1] S. Xu, S. Liu, W. Wang, et al. VibeThinker-3B: Exploring the Frontier of Verifiable Reasoning in Small Language Models. arXiv:2606.16140, 2026.
- [2] S. Zhang, Y. Dong, J. Zhang, et al. Nemotron-Research-Tool-N1: Exploring Tool-Using Language Models with Reinforced Reasoning. arXiv:2505.00024, 2025.
- [3] B. Jin, H. Zeng, Z. Yue, et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516, 2025.
- [4] L. Feng, et al. Group-in-Group Policy Optimization for LLM Agent Training. NeurIPS, 2025.
- [5] G. Dong, et al. Agentic Reinforced Policy Optimization. arXiv:2507.19849; ICLR, 2026.
- [6] J. Feng, S. Huang, X. Qu, et al. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. arXiv:2504.11536, 2025.
- [7] W. Sun, et al. Scaling Long-Horizon LLM Agent via Context-Folding. arXiv:2510.11967, 2025.
- [8] D. Guo, D. Yang, H. Zhang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.
- [9] Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.
- [10] Z. Liu, T. Hoang, J. Zhang, et al. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets (xLAM). NeurIPS, 2024.
- [11] W. Liu, X. Huang, X. Zeng, et al. ToolACE: Winning the Points of LLM Function Calling. ICLR, 2025.
- [12] G. Sheng, C. Zhang, et al. HybridFlow / verl: A Flexible and Efficient RLHF Framework. 2024. github.com/verl-project/verl.
- [13] B. Hui, et al. Qwen2.5-Coder Technical Report. arXiv:2409.12186, 2024.